PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 02 -

DATADOG - INSTRUMENTANDO APM

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 24
REFERÊNCIAS 25

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Nesta aula, você vai descobrir como o tracing distribuído resolve o “problema da caixa-preta” em arquiteturas de microsserviços. Vamos explorar, de forma prática, como um *trace* é formado, como *spans* contam a história completa de uma requisição, como interpretar flame graphs e como o Datadog aplica na prática os conceitos introduzidos pelo Dapper. Você também entenderá como o *trace_id* viaja entre serviços (injeção e extração automática) e como isso permite reconstruir a jornada ponta-a-ponta de qualquer chamada.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Neste laboratório, você aprenderá a instrumentar uma aplicação .NET para geração automática de traces, permitindo a visualização detalhada da jornada de uma requisição através da interface do Datadog. Utilizando ferramentas como o Trace Explorer e o Flame Graph, você será capaz de analisar root spans, spans filhos e latências, identificando gargalos reais por meio de padrões visuais e criando spans customizados para enriquecer o monitoramento com dados de negócio.

Além disso, a aula focará na observabilidade avançada, demonstrando como o Datadog correlaciona erros automaticamente, integrando stacktraces e tags de erro à propagação de falhas entre serviços. Você simulará um fluxo complexo entre múltiplos serviços para verificar, na prática, como ocorre a propagação automática de contexto via headers (traceparent), garantindo a continuidade da observação em arquiteturas distribuídas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Tracing distribuído

O problema da caixa-preta em microsserviços

A adoção de microsserviços revolucionou a engenharia de software ao oferecer modularidade, escalabilidade e resiliência. Porém, ao distribuir funcionalidades em dezenas de serviços independentes, surgiram novos desafios — especialmente o da **observabilidade**. No modelo monolítico, entender o caminho de uma requisição é simples: tudo acontece dentro de um único processo, e ferramentas locais de profiling e logs costumam bastar. Já no ecossistema distribuído, cada requisição pode atravessar uma teia complexa de serviços, bancos, filas e integrações externas, tornando o sistema uma verdadeira “caixa-preta”.

Esse cenário impacta diretamente o dia a dia das equipes. Investigar uma falha ou lentidão deixa de ser um exercício objetivo e passa a se parecer com uma análise forense: mapear manualmente logs de múltiplos serviços, identificar onde a latência surgiu, entender quem chamou quem.

É nesse ponto que o **tracing distribuído** se torna indispensável. Ao propagar um identificador único (Trace ID) em cada salto da requisição, ele reconstrói automaticamente sua jornada completa, do API Gateway ao último serviço chamado. Isso permite visualizar a sequência de operações, a latência de cada etapa, as dependências envolvidas e a raiz de erros ou lentidão. Em outras palavras: o tracing distribuído abre a caixa-preta dos microsserviços e transforma um ecossistema fragmentado em um sistema transparente, inteligível e altamente observável.

O que é um trace na prática (jornada da requisição)

Na teoria, um *trace* (ou rastreamento) é a representação da jornada completa de uma requisição através de um sistema distribuído. Na prática, um *trace* é uma coleção de eventos, chamados **spans**, que são correlacionados por um identificador único, o **Trace ID**. Cada *span* representa uma unidade de trabalho ou uma operação específica dentro do sistema, como uma chamada de banco de dados, uma requisição HTTP a outro serviço, ou a execução de um bloco de código de negócio. A beleza do *trace* reside na sua capacidade de organizar esses *spans* de forma hierárquica e cronológica, contando a história completa de como uma requisição foi processada.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
Trace ID: 12345

  [Span A: POST /cart/add] (Frontend) ---------------------------------------->
      |-- [Span B: HTTP POST] (Frontend -> Cart) ----------------------------->
          |-- [Span C: process_add_item] (Cart) ------------------------->
              |-- [Span D: HTTP GET] (Cart -> Inventory) --------------->
                  |-- [Span E: check_stock] (Inventory) ------------>
                      |-- [Span F: DB Query] (Inventory) ------->
```

Figura 1 - Trace
Fonte: Elaborado pelo autor (2026)

Esta visualização, frequentemente apresentada como um *flame graph* ou *waterfall chart* em ferramentas como o Datadog, é a materialização da jornada da requisição. Ela permite analisar não apenas o tempo total, mas também a contribuição de cada serviço e operação para a latência total. Se o *span* F (a consulta ao banco de dados) for desproporcionalmente longo, encontramos nosso gargalo. Se a chamada do *span* D falhar, saberemos exatamente qual serviço (Cart) não conseguiu se comunicar com qual outro serviço (Inventory) e em que ponto da transação geral isso ocorreu. Assim, o *trace* transforma a investigação de problemas de um exercício de adivinhação para uma análise de dados precisa e direcionada.

Conceitos do Paper Dapper aplicados no Datadog

Em 2010, o Google publicou um paper seminal intitulado “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure”. Neste paper, eles descreveram o sistema que construíram internamente para depurar a performance de seus sistemas massivamente distribuídos. Os conceitos introduzidos pelo Dapper formam a base de quase todos os sistemas de tracing modernos, incluindo o Datadog APM e o padrão aberto OpenTelemetry.

Os conceitos de design do Dapper eram claros e continuam relevantes:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Baixo Overhead	O sistema de tracing não deve impactar significativamente a performance da aplicação que está sendo monitorada.
Transparência para a Aplicação	Os desenvolvedores não deveriam precisar modificar extensivamente seu código para que o tracing funcione.
Escalabilidade	O sistema deve ser capaz de lidar com o volume de dados gerado por uma empresa do tamanho do Google.

Tabela 1 - Design do Dapper
Fonte: Elaborado pelo autor (2026)

Spans automáticos vs spans customizados do negócio

Uma das características mais poderosas das plataformas de APM modernas, como o Datadog, é a capacidade de fornecer visibilidade imediata sobre a performance de uma aplicação através da instrumentação automática. No entanto, para alcançar uma observabilidade verdadeiramente profunda e alinhada ao negócio, é crucial complementar a instrumentação automática com a criação de spans customizados.

Spans Automáticos: A Base da Visibilidade

Os spans automáticos são gerados pelas bibliotecas de tracing do Datadog (como dd-trace-dotnet) sem que seja necessário alterar o código-fonte da aplicação. Ao iniciar a aplicação com o tracer do Datadog ativo, ele utiliza técnicas de “monkey-patching” ou se acopla à Profiling API do .NET para interceptar chamadas a bibliotecas e frameworks comuns. Para cada chamada interceptada, um span é automaticamente criado, medindo sua duração e capturando metadados relevantes.

Em uma aplicação .NET típica, a instrumentação automática irá gerar spans para operações como:

- **Requisições Web de Entrada**: cada requisição recebida por um controller ASP.NET Core se torna o *root span* (ou um *span filho*, se o contexto de trace já existir).
- **Chamadas HTTP de Saída**: qualquer requisição feita através de HttpClient para outros serviços.
- **Consultas a Banco de Dados**: operações executadas via Entity Framework Core ou outros ORMs/drivers suportados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Acessos a Caches**: interações com sistemas como Redis.
- **Processamento de Mensagens**: consumo de mensagens de filas como RabbitMQ ou Kafka.

O grande benefício dos spans automáticos é o **valor imediato com esforço zero**. Apenas habilitando o tracer, obtém-se um mapa detalhado de como os componentes de infraestrutura da aplicação interagem e performam.

Spans Customizados: O Contexto do Negócio

Embora os spans automáticos sejam essenciais, eles têm uma limitação inerente: eles só têm visibilidade sobre as operações de infraestrutura que a biblioteca de tracing foi programada para reconhecer. Eles não têm conhecimento sobre a **lógica de negócio** específica da sua aplicação.

Por exemplo, a instrumentação automática pode mostrar uma requisição ASP.NET Core que levou 500ms para ser executada, mas não consegue detalhar como esses 500ms foram gastos *dentro* do seu código. Foi no cálculo de um preço complexo? Na validação de um conjunto de regras de negócio? Na serialização de um objeto grande?

É aqui que entram os **spans customizados do negócio**. Eles são criados manualmente pelo desenvolvedor, diretamente no código-fonte, para delimitar e medir a duração de blocos de código que representam operações de negócio significativas. Utilizando a API do tracer do Datadog (por exemplo, Datadog.Trace.Tracer.Instance), o desenvolvedor pode “envelopar” uma seção de código em um novo span.

Característica	Spans Automáticos	Spans Customizados do Negócio
Criação	Automática, pelo agente de tracing.	Manual, pelo desenvolvedor via API.
Propósito	Medir a performance de operações de infraestrutura e comunicação entre serviços.	Medir a performance de operações e fluxos de lógica de negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Visibilidade	“Caixa-cinza”: visibilidade nas interações entre componentes conhecidos.	“Caixa-branca”: visibilidade dentro do código da sua aplicação.
Contexto	Técnico (ex: http.method, db.statement).	De negócio (ex: customer.tier, product.id, payment.approved).
Esforço	Baixo (apenas configuração).	Médio (requer modificação de código).
Quando Usar	Sempre, como base da sua estratégia de observabilidade.	Monitorar a performance de algoritmos críticos e fluxos de trabalho complexos.

Tabela 2 – Spans customizados
Fonte: Elaborado pelo autor (2026)

Primeira análise de um trace na UI do Datadog

Após a instrumentação da aplicação e a geração dos primeiros traces, o próximo passo é a análise desses dados na interface de usuário (UI) do Datadog. Assumindo que uma aplicação .NET instrumentada está em execução e gerando tráfego, os traces começarão a aparecer na seção APM > Trace Explorer do Datadog. Ao selecionar um trace específico da lista, você é direcionado para a Trace View, a principal ferramenta para dissecar a jornada de uma requisição.

A Trace View é composta por várias seções interconectadas, mas a visualização central e mais impactante é o Flame Graph.

Cabeçalho do Trace: no topo da visualização, um cabeçalho exibe informações cruciais sobre o trace como um todo:

- **Service e Resource**: o serviço e o recurso do *root span*, indicando o ponto de entrada da requisição (ex: serviço frontend-web, recurso POST /cart/add).
- **Duration**: o tempo total da requisição, do início ao fim.
- **Trace ID**: o identificador único para este trace.
- **Timestamp**: quando a requisição ocorreu.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O Flame Graph: esta é a representação visual da árvore de spans. É uma ferramenta poderosa para entender a latência e a estrutura de uma requisição de relance.

- **Eixo X (Largura)**: representa a duração de um span.
- **Eixo Y (Profundidade)**: representa a pilha de chamadas (call stack).
- **Cores**: representam o serviço onde o span foi executado.

Figura 1: Exemplo de um Flame Graph no Datadog. A largura da barra indica a duração, e a posição vertical indica a hierarquia de chamadas.

```
[IMAGEM: Screenshot da UI do Datadog mostrando um flame graph com anotações indicando o root span, um span filho lar
go (gargalo) e a pilha de chamadas.]
```

Figura 2 – Exemplo de Flame Graph no Datadog
Fonte: Elaborado pelo autor (2026)

Anatomia detalhada de traces e spans

Estrutura interna de um span: IDs, timestamps, tags e erro

Se o *trace* é a história completa da jornada de uma requisição, o **span** é o seu bloco de construção fundamental. Cada span encapsula uma única unidade de trabalho, fornecendo um registro detalhado do que aconteceu, onde aconteceu e quanto tempo levou.

Um span, em sua essência, é um registro de dados estruturado contendo um conjunto de campos padronizados. Embora a implementação exata possa variar ligeiramente entre diferentes ferramentas de tracing, os seguintes componentes são universais e constituem o núcleo de qualquer span, conforme definido por padrões como o OpenTelemetry, que é fortemente suportado pelo Datadog.

- **Trace:** é a “árvore” completa. Todos os spans dentro de um trace compartilham um **Trace ID** único de 64 bits.
- **Span:** é um “galho” da árvore, representando uma única unidade de trabalho com um início e um fim bem definidos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Componente	Descrição	Exemplo em Datadog	Propósito Principal
Trace ID	Um identificador único de 128 bits (representado como uma string hexadecimal de 32 caracteres) que agrupa todos os spans pertencentes a um único trace.	`trace_id: "4f0e...a1b2"`	Correlação: Permite que a plataforma de APM junte todos os pedaços da jornada de uma requisição.
Span ID	Um identificador único de 64 bits (representado como uma string hexadecimal de 16 caracteres) para o span individual dentro do trace.	`span_id: "8c9d...e0f1"`	Identificação: Distingue uma operação específica de todas as outras no mesmo trace.
Parent ID	O Span ID do span que iniciou esta operação. Para o *root span* (o primeiro da requisição), este campo é nulo.	`parent_id: "7b8a...d9c0"`	Causalidade e Hierarquia: Constrói a árvore de chamadas, mostrando qual operação causou qual outra.
Timestamps	Pelo menos dois timestamps de alta precisão: um para o início (`start_time`) e um para o fim (`end_time`) da operação. A diferença entre eles define a **duração** do span.	`start: 167...`, `duration: 52...` (em nanossegundos)	Performance: Mede a latência exata da unidade de trabalho.
Nome (Name)	Uma string legível por humanos que descreve a operação de forma genérica. Em Datadog, isso é frequentemente chamado de **Operation Name**.	`name: "aspnet_core.request"`	Agregação: Agrupa spans semelhantes para calcular métricas agregadas (p. ex., latência média de todas as consultas a um banco de dados).
Tags	Uma coleção de pares chave-valor que adicionam metadados e contexto ao span. As tags podem ser técnicas (adicionadas automaticamente) ou de negócio (adicionadas manualmente).	`tags: {"http.method": "POST", "customer.id": "12345"}`	Contextualização e Filtragem: Permite a busca, filtragem e agrupamento de traces com base em atributos específicos.
Status de Erro (Error Flag)	Um booleano que indica se a operação resultou em um erro. Se `true` , o span também deve conter tags adicionais com detalhes do erro.	`error: 1`	Saúde do Sistema: Identifica rapidamente operações que falharam.

Figura 3 - Estrutura interna de um span
Fonte: Elaborado pelo autor (2026)

Root Span vs Spans filhos

A estrutura hierárquica de um trace, composta por um único *root span* e múltiplos *spans filhos*, é o que permite a reconstrução da lógica de execução de uma requisição. A relação pai-filho entre os spans não é apenas um detalhe técnico; ela é a representação visual e temporal da pilha de chamadas (*call stack*) distribuída através de múltiplos serviços e processos. Interpretar corretamente essa hierarquia é essencial para entender o fluxo de controle, as dependências e a latência em um sistema complexo.

O Root Span: O Ponto de Partida

O **Root Span** é o primeiro span em um trace. Ele é único e especial por duas razões principais:

- **Ponto de Entrada:** ele representa o início da jornada da requisição no sistema monitorado. Geralmente, é criado no primeiro serviço que recebe a requisição externa, como um API Gateway, um *load balancer* ou o serviço de frontend.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Identidade do Trace:** o span_id do *root span* é frequentemente usado como o próprio trace_id em algumas implementações mais antigas, embora o padrão moderno (como o W3C Trace Context) defina um trace_id de 128 bits independente. Mais importante, o *root span* é o único span no trace cujo parent_id é nulo, indicando que ele não tem um chamador conhecido *dentro* do sistema rastreado.

A duração do *root span* encapsula o tempo total de processamento da requisição, de ponta a ponta, do ponto de vista do primeiro serviço. No Flame Graph do Datadog, o *root span* é sempre a barra mais ao topo, da qual todas as outras barras (spans filhos) se originam.

Spans Filhos: A Decomposição do Trabalho

Qualquer span que não seja o *root span* é um Span Filho. Cada span filho representa uma operação que foi causada por outra operação (seu pai). A conexão é estabelecida de forma explícita: o campo parent_id de um span filho contém o span_id de seu span pai.

Essa relação cria uma árvore que reflete a pilha de execução. Quando um serviço A chama um serviço B, o span que representa a operação no serviço B será um filho do span que representa a chamada de saída no serviço A. Isso nos permite visualizar a profundidade da execução:

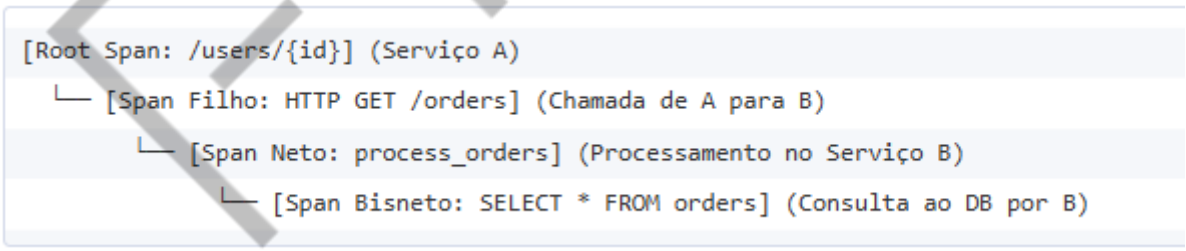


Figura 4 – Span
Fonte: Elaborado pelo autor (2026)

Service Name, Resource Name e Resource cardinality

Dentro da anatomia de um span, três campos de metadados desempenham um papel central na forma como o Datadog agrupa, visualiza e analisa os dados de performance: Service Name, Resource Name e o conceito associado de Resource Cardinality. A configuração correta e a compreensão desses campos são cruciais não

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

apenas para a usabilidade da plataforma, mas também para o controle de custos e a eficácia da monitoração.

O **Service Name** é a tag que identifica a qual aplicação, componente ou microsserviço um span pertence. É o principal pilar para agrupar todos os traces e métricas relacionados a um mesmo bloco de código ou deploy. Em uma arquitetura de microsserviços, cada serviço (ex: authentication-service, product-catalog-service, payment-gateway) deve ter seu próprio e único Service Name.

Este campo é a base do **Unified Service Tagging** do Datadog, que utiliza a tag service (juntamente com env e version) para correlacionar todos os tipos de telemetria — traces, métricas e logs.

Se o Service Name diz “qual serviço” executou o trabalho, o **Resource Name** diz “o que exatamente” foi feito. Ele representa a operação específica que o span está medindo, de uma forma agrupada e genérica. O Resource Name fornece um nível de granularidade mais fino, permitindo analisar a performance de diferentes funcionalidades *dentro* de um mesmo serviço.

Resource Cardinality: O Conceito Crítico

A cardinalidade refere-se ao número de valores únicos que um determinado campo ou tag pode ter. O conceito de Resource Cardinality é um dos mais importantes e frequentemente mal compreendidos no contexto de APM. Para que a agregação e a análise de performance funcionem corretamente, o Resource Name deve ter baixa cardinalidade.

Imagine um endpoint que busca um usuário pelo seu ID: /api/users/{id}. Se o Resource Name for definido com o valor da URL concreta, teríamos recursos como:

- /api/users/123;
- /api/users/456;
- /api/users/789;
- …

Para cada novo usuário, um novo Resource Name único seria criado. Isso é um exemplo de **alta cardinalidade**. As consequências são desastrosas:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Quebra da Agregação: o Datadog não consegue mais agrupar as requisições para calcular a latência média, a taxa de erros ou o throughput do endpoint de “busca de usuário”, pois cada requisição é vista como um recurso completamente diferente.

Inundação da UI: a lista de recursos na UI do Datadog se tornaria infinita e inutilizável, poluída por milhares de entradas únicas.

Impacto no Custo e Performance: a ingestão e indexação de um número massivo de tags únicas podem impactar a performance da plataforma e, em muitos casos, levar a custos maiores de monitoração.

A abordagem correta é usar um Resource Name parametrizado e genérico. Para o exemplo acima, o Resource Name correto seria a definição da rota:

GET /api/users/{id}

Este nome é de **baixa cardinalidade**, pois é o mesmo para todas as requisições que correspondem a este padrão de rota. A informação variável e de alta cardinalidade (como o ID 123) não deve ser descartada, mas sim armazenada em uma **tag** separada, como http.request.path_parameter.id: 123.

Erros no trace: stacktrace, propagação e marcação automática

As bibliotecas de instrumentação automática do Datadog são projetadas para detectar exceções não tratadas em seu código. Quando uma exceção ocorre e não é capturada (ou é capturada e relançada), o tracer intercepta esse evento e automaticamente marca o span ativo naquele momento como um erro.

Essa marcação envolve duas ações principais:

Definir o Status de Erro: o campo error do span é definido como 1 (ou true). Isso sinaliza visualmente o span na UI do Datadog, geralmente colorindo-o de vermelho no Flame Graph, tornando-o imediatamente visível.

Anexar Metadados do Erro: informações cruciais sobre a exceção são adicionadas como tags ao span. As tags padrão incluem:

a) error.msg: a mensagem da exceção (ex: “Object reference not set to an instance of an object.”).

b) error.type: o nome completo do tipo da exceção (ex: System.NullReferenceException).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

c) error.stack: o stack trace completo no momento em que a exceção foi lançada.

O stack trace é, talvez, a peça de informação mais valiosa para um desenvolvedor. Ele aponta para o arquivo exato e o número da linha no código-fonte onde o erro se originou, eliminando qualquer adivinhação sobre a localização do bug.

Propagação de contexto entre serviços

Como o Trace ID "viaja" pela aplicação

A mágica do tracing distribuído — a capacidade de conectar operações discretas em uma narrativa coesa através de múltiplos serviços — depende inteiramente de um mecanismo central: a **propagação de contexto**. Este mecanismo garante que a identidade do trace (o Trace ID) e a informação de causalidade (o Parent Span ID) "viajem" junto com a requisição à medida que ela atravessa as fronteiras dos processos e da rede. Esse processo é realizado através de duas ações complementares: **injeção e extração**.

As bibliotecas de instrumentação automática, como a do Datadog, são projetadas para realizar a injeção e extração de forma transparente para a maioria dos protocolos de comunicação comuns, como HTTP, gRPC e mensagens em filas.

A **injeção** é o processo de pegar as informações de contexto do trace (Trace ID, Parent ID e outras informações, como a decisão de amostragem) e inseri-las em um portador (*carrier*) que será enviado junto com a chamada para o próximo serviço. No contexto de chamadas web, esse portador é quase sempre os **cabeçalhos HTTP** da requisição de saída.

A **extração** é o processo inverso, que ocorre no serviço receptor. Ele consiste em inspecionar a requisição de entrada em busca de cabeçalhos de contexto de trace e, se encontrados, usar essa informação para continuar o trace existente em vez de iniciar um novo.

Instrumentação automática vs. Manual

Executando a aplicação com instrumentação automática (ddtrace-run)

Instrumentação é o processo de adicionar o código de tracing à sua aplicação. O Datadog oferece um espectro de opções, desde a totalmente automática até a totalmente manual, permitindo que você escolha o nível certo de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

esforço e visibilidade.

A instrumentação automática é o ponto de partida e um dos maiores trunfos do Datadog APM. Ela permite obter uma visibilidade profunda sobre o desempenho de uma aplicação .NET sem a necessidade de realizar qualquer alteração no código-fonte. O processo consiste em executar a aplicação em um ambiente onde o .NET Tracer do Datadog está ativo e configurado para se “acoplar” ao processo da aplicação em tempo de execução.

O Datadog .NET Tracer usa a **Profiling API do Common Language Runtime (CLR)**. Esta é uma API de baixo nível que permite que uma biblioteca de profiling (o tracer do Datadog) se anexe ao processo da sua aplicação .NET (dotnet.exe ou w3wp.exe) e reescreva o bytecode (IL - Intermediate Language) de métodos conhecidos em tempo de execução (Just-In-Time compilation).

Quando você habilita o profiler do Datadog (geralmente através de variáveis de ambiente), o CLR carrega a DLL do profiler no seu processo. O profiler então informa ao CLR que ele está interessado em certos métodos (ex: Microsoft.AspNetCore.Hosting.Internal.HostingApplication.ProcessRequestAsync, System.Data.SqlClient.SqlCommand.ExecuteDbDataReader).

Quando um desses métodos está prestes a ser compilado pelo JIT, o profiler intercepta o processo e injeta código IL no início e no fim do método. Esse código injetado é o que inicia e finaliza os spans do Datadog.

Vantagens:

- **Mágica e Abrangente:** não requer *nenhuma* mudança no seu código C#. A instrumentação é verdadeiramente automática para dezenas de bibliotecas populares.
- **Configuração Mínima:** apenas configuração de variáveis de ambiente.
- **Baixo Overhead:** por operar em um nível baixo, é altamente otimizado.

Desvantagens:

- **Visibilidade Limitada:** assim como em outras linguagens, não tem visibilidade sobre a sua lógica de negócio específica. Se você tem um método CalculateComplexReport() que demora 2 segundos, a

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

instrumentação automática só mostrará que o span do endpoint do ASP.NET Core demorou 2 segundos, mas não o que aconteceu dentro dele.

Como funciona o monkey-patching do Datadog

O termo monkey-patching é comumente usado em linguagens dinâmicas como Python e Ruby para descrever a prática de modificar ou estender o comportamento de bibliotecas, módulos ou classes em tempo de execução. Por exemplo, um tracer poderia substituir a implementação original do método send de uma biblioteca HTTP por uma nova versão que inicia um span, chama o método original e depois finaliza o span. Embora o efeito final no .NET seja o mesmo — alterar o comportamento do código existente sem modificar o código-fonte — o mecanismo subjacente é diferente, mais formal e profundamente integrado ao runtime da plataforma.

No ecossistema .NET, a instrumentação automática não é “monkey-patching” no sentido clássico, mas sim um processo sofisticado chamado reescrita de IL (Intermediate Language), habilitado pela CLR Profiling API.

A CLR Profiling API: A Porta de Entrada para o Runtime

A Common Language Runtime (CLR) Profiling API é uma interface poderosa e de baixo nível fornecida pela Microsoft. Ela permite que uma aplicação externa, conhecida como *profiler*, se anexe a um processo .NET e receba notificações detalhadas sobre eventos que ocorrem dentro do CLR. O .NET Tracer do Datadog é, em sua essência, um profiler. Ao iniciar uma aplicação .NET com a variável de ambiente CORECLR_ENABLE_PROFILING=1 definida, você está instruindo o CLR a carregar e dar controle a esse profiler.

O profiler do Datadog utiliza essa API para um propósito específico: interceptar o momento exato em que um método é compilado e modificar seu código antes que ele seja executado.

Quando a instrumentação automática NÃO basta (custom spans)

Confiar exclusivamente na instrumentação automática deixa pontos cegos significativos na sua observabilidade. Ela responde ao “o quê” e “onde” em um nível de infraestrutura, mas frequentemente falha em explicar o “porquê” do ponto de vista da lógica de negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Existem cenários cruciais onde a instrumentação automática, por si só, não é suficiente, tornando o uso de **spans customizados** (instrumentação manual) não apenas benéfico, mas essencial.

Visibilidade Dentro da Lógica de Negócio

O limite mais fundamental da instrumentação automática é que ela não tem conhecimento do seu domínio de negócio. Considere um endpoint de API que leva 800ms para responder. O trace automático pode mostrar que 50ms foram gastos em uma consulta ao banco de dados e 100ms em uma chamada para outro serviço. Mas onde foram gastos os 650ms restantes? Esse tempo foi consumido **dentro do seu código**: processando dados, executando algoritmos complexos, realizando validações, etc. Para a instrumentação automática, esses 650ms são uma “caixa-preta” dentro do seu serviço.

Solução: use spans customizados para “abrir” essa caixa-preta. Ao envolver seções críticas da sua lógica de negócio em spans nomeados, você pode decompor esse tempo e entender exatamente qual parte do seu algoritmo é a mais lenta.

Rastreamento de Protocolos de Comunicação Não Padrão

A instrumentação automática funciona perfeitamente para protocolos padrão como HTTP, gRPC e sistemas de mensageria populares. No entanto, se seus serviços se comunicam através de um meio não convencional, a propagação de contexto automática falhará, resultando em traces quebrados.

Exemplos comuns incluem:

- **Usar um banco de dados como fila:** um serviço escreve um trabalho em uma tabela, e um worker o lê posteriormente.
- **Comunicação via sistema de arquivos:** um serviço gera um arquivo que é processado por outro.
- **Protocolos TCP/UDP customizados.**

Solução: a instrumentação manual é a única maneira de conectar os traces nesses cenários. O desenvolvedor deve:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Injetar o contexto manualmente:** no serviço produtor, usar a API do tracer para extrair o contexto de trace e inseri-lo no payload da comunicação (ex: em uma coluna da tabela do banco de dados ou nos metadados do arquivo).
- **Extrair o contexto manualmente:** no serviço consumidor, ler o contexto do payload e usar a API do tracer para iniciar um novo span como filho do trace original, reconstruindo a cadeia.

Monitoramento de Processos em Background e Funções Assíncronas

Embora o tracer do .NET lide bem com o async/await padrão, ele pode perder o contexto em cenários de threading mais complexos ou em trabalhos que não são iniciados por uma requisição web (ex: um serviço de background que executa tarefas agendadas, como o BackgroundService do .NET).

Solução: para garantir que essas operações sejam rastreadas, você deve iniciar o trace manualmente. No início do seu trabalho em background, crie um novo *root span* para representar aquela execução. Isso garante que qualquer chamada a banco de dados ou serviço HTTP feita por esse trabalho seja capturada em um trace coeso.

Enriquecimento de Traces com Metadados de Negócio

Talvez o uso mais poderoso da instrumentação manual seja a capacidade de adicionar contexto de negócio aos seus traces. A instrumentação automática fornece tags técnicas (http.method, db.instance), mas não pode saber sobre seus conceitos de negócio.

Você quer responder perguntas como:

- “A latência é maior para clientes do plano ‘Enterprise’?”
- “Quais produtos estão mais associados a erros de checkout?”
- “O novo algoritmo de recomendação (feature flag ‘new-algo:true’) é mais rápido que o antigo?”

Solução: use a API span.SetTag() para adicionar essas informações cruciais aos seus spans. Você pode adicionar tags a spans criados automaticamente (acessando o Tracer.Instance.ActiveScope) ou a spans que você mesmo criou. Isso transforma seus traces de um log técnico em uma rica fonte de dados de inteligência de negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Unified Service Tagging

Por que UST é obrigatório para qualquer stack observável

À medida que os sistemas se tornam mais complexos e distribuídos, a quantidade de dados de telemetria (métricas, traces e logs) gerada explode exponencialmente. Cada peça de telemetria, por si só, oferece uma visão limitada. Uma métrica de alta utilização de CPU nos diz que um host está sobrecarregado, mas não por quê. Um log de erro nos diz que algo falhou, mas não o impacto que essa falha teve em outras partes do sistema. Um trace nos mostra a jornada de uma requisição, mas sem contexto, é apenas uma coleção de tempos de execução.

O verdadeiro poder da observabilidade não reside em cada um desses pilares isoladamente, mas na capacidade de **correlacioná-los**. É a capacidade de saltar de um pico em um gráfico de latência (métrica) para os traces exatos que contribuíram para essa latência, e desses traces para os logs detalhados que explicam o erro ocorrido. Essa correlação fluida é o que permite um diagnóstico rápido e eficaz. E o mecanismo que torna essa correlação possível e escalável é o **Unified Service Tagging (UST)**.

O Unified Service Tagging é uma convenção de marcação (tagging) padronizada pelo Datadog que utiliza três tags reservadas para unificar todos os dados de telemetria:

- **env**: o ambiente onde a aplicação está rodando (ex: prod, staging, dev).
- **service**: o nome do serviço, aplicação ou componente que está emitindo a telemetria (ex: authentication-service, frontend-web).
- **version**: a versão do código que está em execução (ex: 2.5.1, ou um hash de commit do git como a8e1b3f).

Adotar o UST não é apenas uma “boa prática”; é um requisito **obrigatório** para qualquer organização que busca construir uma stack de observabilidade madura e eficaz. Sem o UST, cada equipe pode inventar seu próprio esquema de tagging (environment vs env, app vs service), levando a uma confusão generalizada, dashboards quebrados e uma incapacidade de realizar análises cross-funcionais.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Como adicionar UST em diferentes tipos de telemetria

Para que o Unified Service Tagging (UST) seja eficaz, as tags env, service e version devem ser aplicadas de forma consistente em todos os tipos de telemetria emitidos por uma aplicação. O método para aplicar essas tags varia dependendo da fonte da telemetria (APM, Logs, Métricas) e do ambiente de hospedagem (contêineres, VMs, etc.). A estratégia do Datadog é “configure uma vez, aplique em todos os lugares”, utilizando variáveis de ambiente como a principal fonte da verdade.

A Fonte da Verdade: Variáveis de Ambiente

A abordagem mais comum e recomendada para configurar o UST é através das seguintes variáveis de ambiente no host ou contêiner onde a aplicação está rodando:

- DD_ENV: Define a tag env.
- DD_SERVICE: Define a tag service.
- DD_VERSION: Define a tag version.

Quando essas variáveis são definidas, os vários componentes do Datadog (Agent, APM tracers, bibliotecas de logging) as detectam e as aplicam automaticamente aos dados que coletam.

Aplicando UST em Traces (APM)

Esta é a aplicação mais direta. A biblioteca de tracin lê automaticamente as variáveis de ambiente DD_ENV, DD_SERVICE e DD_VERSION no momento em que a aplicação é iniciada. Cada span gerado por essa instância do tracer será automaticamente marcado com essas três tags. Nenhuma configuração adicional no código é geralmente necessária.

Aplicando UST em Logs

A forma ideal de correlacionar logs e traces é habilitando a **injeção automática de logs pelo tracer**, que adiciona às entradas de log as tags **env**, **service**, **version**, além de **trace_id** e **span_id**, permitindo correlação perfeita no Datadog. O tracer ajusta automaticamente bibliotecas como Serilog e NLog, e a funcionalidade geralmente é ativada via `DD_LOGS_INJECTION=true`. Caso isso não seja possível, o

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog Agent pode aplicar as tags durante a coleta dos logs, herdando metadados do host ou contêiner.

Aplicando UST em Métricas

As métricas podem vir de várias fontes, e a aplicação do UST depende de cada uma.

Métricas de Integração (ex: ASP.NET Core, SQL Server): as métricas coletadas automaticamente pelas integrações do Datadog Agent herdam as tags do host ou contêiner onde o Agent está rodando. Se o seu contêiner da aplicação tem a label service: user-profile-service, as métricas de CPU e memória daquele contêiner também terão essa tag.

Métricas Customizadas (via DogStatsD): é o serviço, geralmente embutido no Agent, que recebe métricas customizadas via UDP. Ao enviar uma métrica do seu código, você pode (e deve) adicionar as tags UST.

Impacto prático no APM e Service Map

O Service Map é uma visualização em tempo real que mostra seus serviços como nós em um grafo e as chamadas entre eles como arestas. Ele é projetado para responder a perguntas fundamentais como: “Quais serviços se comunicam com quais outros?”, “Qual é a saúde (latência, taxa de erro, throughput) de cada dependência?” e “Onde estão os gargalos na comunicação entre serviços?”.

Impacto nas Funcionalidades do APM

O UST é a espinha dorsal que sustenta a maioria das funcionalidades avançadas do APM.

Service e Resource Pages: as páginas de resumo de Serviço (Service Page) e Recurso (Resource Page) no Datadog agregam dados de performance (latência p50/p90/p99, taxa de requisições, taxa de erro) para um serviço ou um endpoint específico. Essa agregação só é possível porque o Datadog pode agrupar todos os spans que compartilham o mesmo service e resource name. Sem o UST, essas páginas ficam vazias ou mostram dados incorretos.

Análise de Deployments (Tag version): a tag version é o que alimenta a capacidade do Datadog de analisar o impacto de um novo deploy. Na Service Page, você pode ver sobreposto aos gráficos de latência e erro marcadores que indicam

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

quando uma nova versão foi implantada. Isso permite correlacionar visualmente uma mudança no comportamento do sistema com uma mudança no código.

- **Sem a tag version:** você vê um pico de latência, mas não sabe o que o causou. Foi um deploy? Um aumento no tráfego? Um problema de infraestrutura?
- **Com a tag version:** você vê o pico de latência alinhado perfeitamente com o marcador da version: 2.1.5. O diagnóstico é imediato: o último deploy introduziu uma regressão de performance. Você pode então filtrar todos os traces para essa versão específica para investigar mais a fundo.

Análise por Ambiente (Tag env): a tag env é crucial para o gerenciamento do ciclo de vida do software e para o controle de alertas. Ela permite:

- **Separar Dados:** analisar a performance em prod sem o ruído dos ambientes de dev e staging.
- **Configurar Monitores por Ambiente:** criar alertas de alta severidade apenas para problemas que ocorrem em env:prod, enquanto alertas para env:staging podem ter uma prioridade mais baixa.
- **Comparar Performance:** comparar o comportamento de uma nova funcionalidade em staging com a linha de base em prod antes de liberá-la para todos os usuários.

Correlação entre Traces, Métricas e Logs: como mencionado anteriormente, o UST é o elo perdido que conecta os três pilares da observabilidade. Quando você está olhando para um trace e quer ver os logs detalhados daquele exato momento, o Datadog usa as tags env, service e version (junto com trace_id) para buscar e exibir apenas os logs relevantes. Sem essa marcação consistente, essa funcionalidade de “clique para correlacionar” falha, forçando o engenheiro a voltar para a busca manual de logs, perdendo tempo valioso durante um incidente.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o desafio da "caixa-preta" em arquiteturas de microsserviços, onde a distribuição de chamadas torna a identificação da causa raiz uma tarefa complexa. Compreendemos o tracing distribuído como a solução fundamental, definindo o trace como a jornada completa de uma requisição e o span como sua unidade básica de trabalho. Estudamos a estrutura interna desses componentes — envolvendo IDs, hierarquia pai-filho e tags — e as bases teóricas do paper Dapper, que estabeleceu princípios como baixo overhead, amostragem e transparência, agora aprimorados por plataformas como o Datadog.

Aprofundamos a análise técnica ao diferenciar spans automáticos de spans customizados de negócio, além de dominar conceitos críticos como Resource Cardinality e seu impacto em custos e agregação de métricas. Aprendemos a interpretar visualmente o desempenho por meio de Flame Graphs, identificando gargalos em barras largas, paralelismo em ramificações e a falta de instrumentação em lacunas no fluxo. Essa leitura visual é essencial para transformar dados brutos em inteligência operacional e otimização de sistemas.

Por fim, estudamos o mecanismo de propagação de contexto, que utiliza a injeção e extração de metadados para conectar chamadas entre múltiplos serviços (A → B → C) em uma narrativa única e contínua. Vimos como o sistema detecta e propaga erros automaticamente, capturando mensagens, tipos de erro e stacktraces para garantir que falhas sejam rastreadas em toda a cadeia de dependências. Esse conjunto de ferramentas e conceitos consolida a base necessária para a observabilidade moderna em ambientes distribuídos.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

DATADOG. **Datadog APM Documentation – Distributed Tracing, Spans, Service Map and Instrumentation.** Datadog, 2025. Disponível em: https://docs.datadoghq.com/tracing/. Acesso em: 08 jan. 2026.

GOOGLE INC. **Dapper, a Large-Scale Distributed Systems Tracing Infrastructure (Google Technical Report No. dapper-2010-1)**. Mountain View, CA: Google, 2010. Disponível em: https://research.google/pubs/pub36356/. Acesso em: 08 jan. 2026.

OPENTELEMETRY PROJECT. **OpenTelemetry Specification – Traces, Span Model and Context Propagation.** Cloud Native Computing Foundation (CNCF), 2025. Disponível em: https://opentelemetry.io/docs/concepts/signals/traces/. Acesso em: 08 jan. 2026.

SIGELMAN, Benjamin H. et al. **Dapper: a Large-Scale Distributed Systems Tracing Infrastructure**. Google Research, 2010. Disponível em: https://research.google/pubs/pub36356/. Acesso em: 08 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Agent, Host, Container.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com